

Database

Contents

Module	Title	Main focus
1	Introduction to Data and Databases	What data is, why databases exist, and what a DBMS does
2	Database Fundamentals	Tables, rows, columns, data types, NULL, primary keys, and foreign keys
3	Relational Database Concepts	Entities, attributes, relationships, cardinality, and ER diagrams
4	Introduction to SQL	What SQL is and how basic SQL statements are written
5	Creating Database Structures (DDL)	CREATE, ALTER, DROP, TRUNCATE, and table constraints
6	Manipulating Data (DML)	INSERT, UPDATE, DELETE, bulk inserts, and basic transactions
7	Retrieving Data	SELECT, WHERE, operators, DISTINCT, LIKE, ORDER BY, and LIMIT
8	SQL Functions	String, numeric, date, and aggregate functions
9	Grouping Data	GROUP BY, HAVING, and aggregation mistakes to avoid

Module 1: Introduction to Data and Databases

Learning goals

- Define data and information using everyday examples.
- Explain why storing data carefully is important.
- Describe the problems of file-based or spreadsheet-only systems.
- Explain the difference between a database and a DBMS.
- Recognize common database systems used in real life.

1.1 What is data?

Data means raw facts. A fact can be a number, a word, a date, a picture, a sound, or a measurement. By itself, a single piece of data may not tell a complete story. For example, the number 88 is data, but we do not know what it means until we add context.

Raw data	Context added	Information created
88	Amina scored 88 in Mathematics	Amina performed well in Mathematics
2026-01-15	This is the exam date	The exam happened on 15 January 2026
Dhaka	This is a student city	The student lives in Dhaka
Level 9	This is the class level	The student is studying in level 9

Definition

Data is raw fact. Information is data that has been organized or explained so that it becomes meaningful.

1.2 Why do people store data?

People store data because memory is limited and because organizations need accurate records. A school stores student names, marks, attendance, fees, and contact numbers. A hospital stores patient details, appointments, test results, and prescriptions. A shop stores products, sales, suppliers, and payments.

Stored data helps people answer questions such as:

- Which students are in level 9?
- Which products are almost out of stock?
- How many patients visited the clinic this month?
- Which customers still need to pay?
- What was the average score in Mathematics?

1.3 Storing data before databases

Before learning databases, imagine a school that stores student records in notebooks, separate paper forms, or many spreadsheet files. This can work when there are only a few students. However, as the amount of data grows, problems appear.

Problem	Simple example	Why it is harmful
Duplication	The same student phone number is written in five files	A change must be made in five places
Inconsistency	One file says "Karim Hasan" and another says "K. Hassan"	People may think these are two different students
Difficult searching	The teacher must open many files to find one score	Work becomes slow and error-prone
Weak security	Anyone with the spreadsheet can see all records	Private information may be exposed
Poor backup	A file is accidentally deleted	Important data may be lost

Example: the address problem

A student changes address. If the address is stored in a class list, a fee sheet, an exam sheet, and an attendance sheet, someone must update every file. If one file is missed, the school now has two different addresses for the same student. A database helps reduce this problem by storing important facts in a controlled way.

1.4 What is a database?

A database is an organized collection of related data. Organized means the data is not thrown together randomly. Related means the data belongs to the same purpose or system. A school database may store students, courses, teachers, attendance, and scores because all of these are related to school management.

A database is not only a table. A database can contain many tables, rules, saved queries, security settings, and other objects. At the beginner level, we usually start by thinking about tables because tables are easy to see and understand.

1.5 What is a DBMS?

DBMS stands for Database Management System. It is software that helps users create, store, protect, update, and retrieve data from a database. The DBMS is like a librarian. A library has books, but the librarian helps organize them, find them, lend them safely, and keep records. In the same way, a DBMS manages the database.

Term	Meaning	Analogy
Database	The organized collection of data	The books and records in a library
DBMS	The software that manages the database	The librarian and library system
Database application	A program or interface users work with	The online library search screen

1.6 Popular DBMS examples

Different organizations use different DBMS software depending on size, cost, performance needs, and developer preference. The ideas in this document apply to most relational database systems, even if some SQL commands differ slightly.

DBMS	Common use
SQLite	Small apps, learning, mobile apps, simple local databases
MySQL	Websites, applications, and many business systems
PostgreSQL	Applications that need strong features and standards support
Microsoft SQL Server	Business systems, especially in Microsoft environments
Oracle Database	Large enterprise systems, banks, telecom, and government systems

1.7 What makes databases useful?

- They keep data organized in a consistent structure.
- They allow fast searching and filtering.
- They reduce unnecessary duplication when designed well.
- They can protect data with users, passwords, and permissions.
- They can handle many users working at the same time.
- They can keep data safer through backup and recovery tools.

Key takeaways

- Data is raw fact; information is meaningful data.
- A database stores related data in an organized way.
- A DBMS is software used to manage a database.
- File-based systems can suffer from duplication, inconsistency, slow searching, and weak security.

Module 2: Database Fundamentals

Learning goals

- Understand tables, rows, and columns.
- Choose simple data types for common values.
- Explain NULL, primary keys, and foreign keys.
- Understand why rules called constraints protect data quality.

2.1 Tables, rows, and columns

A table stores data about one kind of thing. For example, a students table stores data about students. A row stores data about one individual item in the table. A column stores one kind of fact for every row.

student_id	first_name	last_name	level	city
1	Amina	Rahman	9	Dhaka
2	Karim	Hasan	9	Chittagong
3	Rina	Akter	10	Dhaka
4	Mehedi	Islam	10	Sylhet

Example table: students

Word	Meaning in the table above
Table	The whole students structure
Row or record	One complete student, such as Amina Rahman
Column or field	One type of fact, such as city or level
Cell	One value in one row and one column, such as Dhaka

2.2 Data types

A data type tells the database what kind of value a column should store. This is important because the database handles numbers, text, dates, and true/false values differently. For example, the number 100 can be added to another number. The text "100" looks similar, but the database may treat it as words, not as a number.

Data type	Used for	Example value	Beginner note
Integer	Whole numbers	9, 150, -3	Good for counts, IDs, ages, class levels
Decimal	Numbers with fractions	85.50, 3.75	Good for prices, averages, percentages
String or text	Letters and symbols	Amina, Dhaka	Good for names, emails, addresses
Boolean	True or false	TRUE, FALSE	Good for yes/no facts such as is_active
Date	Calendar dates	2026-01-15	Good for birthdays and exam dates
Timestamp	Date and time together	2026-01-15 09:30:00	Good for logins and order times

Important

A phone number is usually stored as text, not as an integer. It may start with zero, include a country code, or contain symbols such as +. You normally do not add phone numbers together, so numeric calculation is not the goal.

2.3 NULL values

NULL means missing, unknown, or not available. NULL is not the same as zero. NULL is also not the same as an empty text value. For example, if a student's email address is not known yet, the email column may contain NULL.

Value	Meaning
0	A known number with value zero

Value	Meaning
Empty text	A known text value that contains no characters
NULL	No value is currently known or stored

Example

If a score is 0, the student received zero marks. If a score is NULL, the score has not been entered yet or is unknown. These two meanings are very different.

2.4 Primary keys

A primary key is a column, or a combination of columns, that uniquely identifies each row in a table. Unique means no two rows have the same primary key value. A primary key is important because names are not always unique. Two students may have the same name, but they should not have the same `student_id`.

<code>student_id</code>	<code>first_name</code>	<code>last_name</code>
1	Amina	Rahman
2	Amina	Rahman

In the example above, the names are the same, but `student_id` values are different. Therefore the database can still tell the two students apart.

2.5 Foreign keys

A foreign key is a column in one table that refers to the primary key of another table. Foreign keys are used to connect related tables. For example, a scores table can store `student_id` to show which student received each score.

Students table

<code>student_id</code>	Student
1	Amina Rahman

Scores table

<code>score_id</code>	<code>student_id</code>	subject	score
101	1	Mathematics	88

The scores table does not need to repeat Amina's full name for every score. It can store `student_id`, and that `student_id` points back to the correct row in the students table.

2.6 Relationships between tables

Tables are often related. A student can have many scores. A score belongs to one student. This is a relationship. Relationships help a database store data without repeating the same information again and again.

Beginner checkpoint

Do not worry if relationships feel abstract at first. For now, remember that a primary key identifies a row, and a foreign key points to a row in another table. Module 3 explains relationships more carefully.

2.7 Constraints

A constraint is a rule that the database enforces to protect data quality. For example, a primary key constraint prevents duplicate IDs. A NOT NULL constraint prevents an important column from being left empty.

Constraint	What it prevents
PRIMARY KEY	Duplicate or missing identity values for rows
FOREIGN KEY	References to rows that do not exist
NOT NULL	Missing values in important columns
UNIQUE	Duplicate values in a column that must be different
CHECK	Values that break a logical rule, such as score > 100
DEFAULT	Missing values when a safe automatic value can be used

Key takeaways

- A table stores one kind of data; a row stores one item; a column stores one type of fact.
- Data types tell the database what kind of values are allowed.
- NULL means missing or unknown, not zero.
- Primary keys identify rows; foreign keys connect rows in different tables.
- Constraints are rules that protect data accuracy.

Module 3: Relational Database Concepts

Learning goals

- Explain what a relational database is.
- Identify entities and attributes in a real-world situation.
- Recognize one-to-one, one-to-many, and many-to-many relationships.
- Read and draw simple ER diagrams.
- Convert a simple design idea into tables.

3.1 What is a relational database?

A relational database stores data in tables that can be related to each other. The word relational comes from relation, which is another formal word for a table. In beginner terms, a relational database is a database made of tables that are connected using keys.

Instead of keeping all data in one huge table, a relational database separates data into smaller, focused tables. This makes the data easier to update, protect, and understand.

3.2 Entities and attributes

An entity is something we want to store data about. An attribute is a fact about that entity. In a database table, an entity often becomes a table, and attributes often become columns.

Real-world idea	Entity	Possible attributes
A school keeps student records	Student	student_id, first_name, last_name, level, city
A library keeps book records	Book	book_id, title, author, publication_year
A shop keeps product records	Product	product_id, product_name, price, stock_quantity
A clinic keeps appointment records	Appointment	appointment_id, patient_id, doctor_id, appointment_date

3.3 Relationships

A relationship describes how entities are connected. For example, a student receives scores. A teacher teaches courses. A customer places orders. When designing a database, you should ask: which things are connected, and how many of one thing can be connected to the other?

3.4 Cardinality

Cardinality describes the number of possible connections between two entities. The three basic relationship types are one-to-one, one-to-many, and many-to-many.

Relationship type	Meaning	Example
One-to-one	One row in table A is related to one row in table B	One person has one passport record
One-to-many	One row in table A can be related to many rows in table B	One student can have many scores
Many-to-many	Many rows in table A can relate to many rows in table B	Students can enroll in many courses; courses can have many students

Important: many-to-many needs a linking table

A many-to-many relationship is usually handled by creating a third table. For students and courses, the linking table could be called enrollments. It stores `student_id` and `course_id` to record which student joined which course.

3.5 Entity Relationship diagrams

An Entity Relationship diagram, usually called an ER diagram or ERD, is a picture of a database design. It shows entities, attributes, and relationships before the database is built. A simple ERD helps everyone agree on the structure before writing SQL.

Student
student_id (PK)
first_name
last_name
level
city

Score
score_id (PK)
student_id (FK)
subject
score
exam_date

In this diagram, one Student can have many Score records. Each Score record belongs to one Student. The students table uses `student_id` as its primary key. The scores table stores `student_id` as a foreign key.

3.6 Designing a simple database

When designing a database, start with questions. What do we need to remember? What questions do users need to answer? What things are involved? What facts describe those things? How are the things connected?

1. Write the purpose of the database. Example: store students and exam scores.
2. List the entities. Example: Student and Score.
3. List the attributes for each entity. Example: a Student has `first_name`, `last_name`, `level`, and `city`.
4. Choose a primary key for each entity. Example: `student_id` and `score_id`.
5. Identify relationships. Example: one student can have many scores.
6. Decide where foreign keys should go. Example: `scores.student_id` points to `students.student_id`.

Example design decision

Should `subject` be a text column in `scores`, or should there be a separate `subjects` table? For a very small beginner database, `subject` as text can be acceptable. For a larger school system, a separate `subjects` table is better because it avoids spelling differences such as "Math", "Maths", and "Mathematics".

3.7 From design to tables

Design idea	Database table result
Entity: Student	Create a table named students
Attribute: first_name	Create a column named first_name
Primary key: student_id	Make student_id unique and required
Relationship: Student has many Scores	Put student_id as a foreign key in scores
Rule: score must be 0 to 100	Add a CHECK constraint to scores.score

Key takeaways

- A relational database stores data in related tables.
- Entities are things we store data about; attributes are facts about those things.
- Cardinality tells us how many rows can be connected between tables.
- ER diagrams help plan a database before writing SQL.
- Many-to-many relationships usually need a linking table.

Module 4: Introduction to SQL

Learning goals

- Explain what SQL is used for.
- Recognize basic SQL statement structure.
- Understand SQL categories such as DDL, DML, and query commands.
- Write readable SQL using simple naming conventions and comments.

4.1 What is SQL?

SQL stands for Structured Query Language. It is the language commonly used to communicate with relational databases. With SQL, you can create tables, add data, update data, delete data, and ask the database questions.

Pronunciation note

Some people say "S-Q-L" letter by letter. Others say "sequel". Both are commonly understood. In class, use whichever pronunciation your teacher prefers.

4.2 What can SQL do?

Need	SQL command idea	Example question
Create structure	CREATE TABLE	How do we make a students table?
Add data	INSERT	How do we add Amina as a student?
Change data	UPDATE	How do we change a city value?
Remove data	DELETE	How do we delete a wrong row?
Read data	SELECT	Which students are in level 9?
Summarize data	GROUP BY with functions	What is the average score per subject?

4.3 SQL statement basics

A SQL statement is an instruction sent to the database. Many SQL statements end with a semicolon. Keywords such as SELECT, FROM, and WHERE are often written in uppercase to make the query easier to read. Table and column names are usually written in lowercase.

```
SELECT first_name, last_name
FROM students
WHERE level = 9;
```


This statement asks the database to return first and last names from the students table, but only for students whose level is 9.

4.4 SQL is usually not case-sensitive for keywords

In many database systems, SQL keywords can be written in uppercase or lowercase. The following two statements often mean the same thing. However, uppercase keywords are easier for beginners to read.

```
SELECT first_name FROM students;
select first_name from students;
```

Caution

Although SQL keywords are often case-insensitive, database names, table names, and text values may behave differently depending on the DBMS. Always follow your class or organization naming rules.

4.5 Comments

A comment is text written for humans, not for the database to execute. Comments help explain what a query is doing. They are useful when queries become longer.

```
-- Show students from level 9
SELECT first_name, last_name
FROM students
WHERE level = 9;

/* This is a longer comment.
   It can cover more than one line. */
```

4.6 Naming conventions

A naming convention is a consistent way of naming tables and columns. Good names make a database easier to understand. Beginners should use names that are simple, descriptive, and consistent.

Good practice	Example
Use lowercase names	students, scores
Use underscores for multiple words	first_name, exam_date
Avoid spaces in names	Use student_id instead of student id
Use plural table names consistently	students, courses, scores
Avoid unclear abbreviations	Use exam_date instead of ex_dt

4.7 Main categories of SQL commands

Category	Meaning	Commands in this document
DDL	Data Definition Language. Used to define database structures.	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language. Used to change the data stored in tables.	INSERT, UPDATE, DELETE
DQL	Data Query Language. Used to retrieve data.	SELECT
TCL	Transaction Control Language. Used to manage transaction results.	COMMIT, ROLLBACK

4.8 Reading SQL slowly

When you are new to SQL, read each line in plain English. Do not panic if the punctuation looks strange. SQL becomes easier when you translate commands into everyday language.

```
SELECT city
FROM students
WHERE first_name = 'Amina';
```

Plain English translation: Show the city from the students table where the first name is Amina.

Key takeaways

- SQL is used to communicate with relational databases.
- SQL statements often end with a semicolon.
- Keywords are commonly written in uppercase for readability.
- Comments explain SQL code to humans.
- Clear naming makes databases easier to use and maintain.

Module 5: Creating Database Structures (DDL)

Learning goals

- Understand the purpose of DDL commands.
- Create a simple table with columns and data types.
- Use basic constraints to protect data quality.
- Understand ALTER TABLE, DROP TABLE, and TRUNCATE TABLE at a beginner level.

5.1 What is DDL?

DDL stands for Data Definition Language. DDL commands create and change the structure of a database. Think of DDL as building the empty shelves and labels before putting books into a library. The structure must exist before data can be stored properly.

DDL command	Purpose
CREATE DATABASE	Creates a new database container
CREATE TABLE	Creates a new table structure
ALTER TABLE	Changes an existing table structure
DROP TABLE	Deletes a table structure and its data
TRUNCATE TABLE	Removes all rows quickly, but keeps the table structure

5.2 Creating a database

Some DBMSs require you to create a database before creating tables. The exact command may differ between systems. In many SQL systems, it looks like this:

```
CREATE DATABASE school_db;
```

DBMS note

SQLite stores a database as a file, so beginners using SQLite may not type CREATE DATABASE. They may simply open or create a database file. Your teacher will tell you which tool is being used.

5.3 Creating a table

A CREATE TABLE statement gives the table name and then lists the columns. Each column normally has a name, a data type, and sometimes a constraint.

```
CREATE TABLE students (
  student_id INTEGER PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  level INTEGER NOT NULL,
  city TEXT,
  email TEXT UNIQUE
);
```

This command creates a students table. student_id is the primary key. first_name, last_name, and level must not be NULL. email must be unique if it is provided. city is allowed to be NULL because a city may not be known yet.

5.4 Understanding the table definition

Part of SQL	Meaning
CREATE TABLE students	Make a new table named students
student_id INTEGER PRIMARY KEY	Create an integer ID that uniquely identifies each student
first_name TEXT NOT NULL	Create a text column that must have a value
email TEXT UNIQUE	Create a text column where duplicate email values are not allowed
Commas between columns	Separate one column definition from the next
Semicolon at the end	Marks the end of the SQL statement

5.5 Creating a table with a foreign key

The scores table stores exam scores. Each score belongs to a student, so the scores table has a student_id column that refers to students.student_id.

```
CREATE TABLE scores (
  score_id INTEGER PRIMARY KEY,
  student_id INTEGER NOT NULL,
  subject TEXT NOT NULL,
  score INTEGER NOT NULL,
  exam_date DATE NOT NULL,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  CHECK (score >= 0 AND score <= 100)
);
```

The foreign key helps prevent a score from being recorded for a student who does not exist in the students table. The CHECK rule helps prevent impossible scores such as -5 or 125.

5.6 Constraints in CREATE TABLE

Constraint	Example	Explanation
PRIMARY KEY	student_id INTEGER PRIMARY KEY	Uniquely identifies each row
FOREIGN KEY	FOREIGN KEY (student_id) REFERENCES students(student_id)	Connects a row to another table
UNIQUE	email TEXT UNIQUE	Prevents duplicate values in that column
NOT NULL	first_name TEXT NOT NULL	Requires a value
CHECK	CHECK (score >= 0 AND score <= 100)	Allows only values that pass a condition
DEFAULT	status TEXT DEFAULT 'active'	Uses an automatic value when none is provided

5.7 ALTER TABLE

ALTER TABLE changes the structure of an existing table. For example, if the school later decides to store student phone numbers, we can add a phone column.

```
ALTER TABLE students
ADD phone TEXT;
```

Think before changing structure

Changing a table after it already contains data can be risky. You should understand what data is already there and how applications use the table before altering it.

5.8 DROP TABLE

DROP TABLE deletes the table structure and the data inside it. This is a dangerous command because the table is removed. In real systems, you should back up data and check carefully before dropping tables.

```
DROP TABLE scores;
```

5.9 TRUNCATE TABLE

TRUNCATE TABLE removes all rows from a table while keeping the table structure. It is usually faster than deleting rows one by one, but it can also be dangerous because it removes all data.

```
TRUNCATE TABLE scores;
```

DBMS note

Not every DBMS supports TRUNCATE TABLE in the same way. SQLite, for example, does not use TRUNCATE TABLE. In SQLite, a similar result can be achieved with DELETE FROM table_name, but that is a DML command.

Key takeaways

- DDL commands define or change database structures.
- CREATE TABLE creates an empty table with named columns and data types.
- Constraints are written into table definitions to protect data quality.
- DROP TABLE removes a table; TRUNCATE TABLE removes all rows but keeps structure where supported.

Module 6: Manipulating Data (DML)

Learning goals

- Use INSERT to add rows to a table.
- Use UPDATE to change existing rows safely.
- Use DELETE to remove rows safely.
- Understand the basic purpose of transactions.

6.1 What is DML?

DML stands for Data Manipulation Language. DML commands work with the data inside tables. If DDL builds the empty table structure, DML fills the table, changes values, and removes rows when needed.

DML command	Purpose
INSERT	Adds new rows
UPDATE	Changes existing rows
DELETE	Removes existing rows

6.2 INSERT: adding one row

INSERT adds a new row to a table. It is a good habit to list the columns you are inserting into. This makes the command clearer and safer.

```
INSERT INTO students (student_id, first_name, last_name, level, city, email)
VALUES (1, 'Amina', 'Rahman', 9, 'Dhaka', 'amina@example.com');
```

Text values and dates are usually written inside single quotes. Numeric values such as 1 and 9 do not need quotes.

6.3 INSERT: adding multiple rows

Some DBMSs allow several rows to be inserted with one INSERT statement. This is useful for sample data and setup scripts.

```
INSERT INTO students (student_id, first_name, last_name, level, city, email)
VALUES
(2, 'Karim', 'Hasan', 9, 'Chittagong', 'karim@example.com'),
(3, 'Rina', 'Akter', 10, 'Dhaka', 'rina@example.com'),
(4, 'Mehedi', 'Islam', 10, 'Sylhet', 'mehedi@example.com');
```

6.4 INSERT into the scores table

Because scores.student_id is a foreign key, the student_id value should already exist in students. The database can reject a score for a missing student.

```
INSERT INTO scores (score_id, student_id, subject, score, exam_date)
VALUES
  (101, 1, 'Mathematics', 88, '2026-01-15'),
  (102, 1, 'English', 79, '2026-01-18'),
  (103, 2, 'Mathematics', 72, '2026-01-15'),
  (104, 3, 'Science', 91, '2026-01-20');
```

6.5 UPDATE: changing existing data

UPDATE changes values in existing rows. The WHERE clause is very important because it controls which rows will change.

```
UPDATE students
SET city = 'Narayanganj'
WHERE student_id = 1;
```

This changes the city for only the student whose student_id is 1. If the WHERE clause is missing, many DBMSs will update every row in the table.

Dangerous mistake

```
UPDATE students SET city = 'Narayanganj';
```

This command has no WHERE clause. It may change the city for every student. Always check the WHERE clause before running UPDATE.

6.6 DELETE: removing rows

DELETE removes rows from a table. Like UPDATE, DELETE should usually have a WHERE clause.

```
DELETE FROM scores
WHERE score_id = 104;
```

This deletes only the score row with score_id 104. Without WHERE, a DELETE statement can remove every row from the table.

6.7 Safe update and delete habit

Before running UPDATE or DELETE, first write a SELECT statement with the same WHERE clause. This lets you see which rows will be affected.

```
-- Step 1: check the row first
SELECT *
FROM students
WHERE student_id = 1;

-- Step 2: update only after checking
UPDATE students
SET city = 'Narayanganj'
WHERE student_id = 1;
```

6.8 Transactions: basic introduction

A transaction is a group of database actions treated as one unit of work. If all actions are correct, the transaction can be saved. If something goes wrong, the transaction can be undone. At beginner level, remember two commands: COMMIT saves the changes, and ROLLBACK cancels the changes.

```

BEGIN TRANSACTION;

UPDATE students
SET city = 'Narayanganj'
WHERE student_id = 1;

-- Save the change permanently
COMMIT;

BEGIN TRANSACTION;

DELETE FROM scores
WHERE student_id = 1;

-- Undo the delete if it was a mistake
ROLLBACK;

```

Why transactions matter

Imagine transferring money between bank accounts. Money should not disappear from one account unless it also appears in the other account. Transactions help keep related changes together.

Key takeaways

- DML commands manipulate rows stored inside tables.
- INSERT adds rows; UPDATE changes rows; DELETE removes rows.
- WHERE is essential for safe UPDATE and DELETE commands.
- Transactions allow changes to be saved with COMMIT or cancelled with ROLLBACK.

Module 7: Retrieving Data with Queries

Learning goals

- Use SELECT to retrieve data from a table.
- Filter rows with WHERE and comparison operators.
- Combine conditions with AND, OR, and NOT.
- Use DISTINCT, BETWEEN, IN, LIKE, ORDER BY, and LIMIT.
- Read simple query results correctly.

7.1 What is a query?

A query is a question or request sent to a database. In SQL, SELECT is the main command used to retrieve data. When you write SELECT, you are asking the database to show certain data from one or more tables. In this module, we query one table at a time.

7.2 Sample data for this module

The examples use these rows. If you practice in a database tool, your results should look similar after inserting this data.

student_id	first_name	last_name	level	city
1	Amina	Rahman	9	Dhaka
2	Karim	Hasan	9	Chittagong
3	Rina	Akter	10	Dhaka
4	Mehedi	Islam	10	Sylhet
5	Farah	Khan	9	Khulna
6	Nabil	Ahmed	10	Dhaka

Sample students table

score_id	student_id	subject	score	exam_date
101	1	Mathematics	88	2026-01-15
102	1	English	79	2026-01-18
103	2	Mathematics	72	2026-01-15
104	2	English	84	2026-01-18
105	3	Science	91	2026-01-20
106	4	Mathematics	65	2026-01-15
107	5	English	92	2026-01-18
108	6	Science	77	2026-01-20

Sample scores table

7.3 SELECT all columns

The asterisk means all columns. This is useful for quick learning, but in real work it is often better to list only the columns you need.

```
SELECT *
FROM students;
```

7.4 SELECT specific columns

To show only certain columns, list them after SELECT and separate them with commas.

```
SELECT first_name, last_name, city
FROM students;
```

first_name	last_name	city
Amina	Rahman	Dhaka
Karim	Hasan	Chittagong
Rina	Akter	Dhaka

Partial result shown

7.5 DISTINCT

DISTINCT removes duplicate values from the result. It does not delete data from the table. It only changes what is shown in the query result.

```
SELECT DISTINCT city
FROM students;
```

If three students live in Dhaka, Dhaka appears only once in the DISTINCT result.

7.6 WHERE

WHERE filters rows. Only rows that pass the condition are returned.

```
SELECT first_name, last_name, level
FROM students
WHERE level = 9;
```

first_name	last_name	level
Amina	Rahman	9
Karim	Hasan	9
Farah	Khan	9

7.7 Comparison operators

Operator	Meaning	Example condition
=	Equal to	level = 9

Operator	Meaning	Example condition
<> or !=	Not equal to	city <> 'Dhaka'
>	Greater than	score > 80
<	Less than	score < 50
>=	Greater than or equal to	score >= 80
<=	Less than or equal to	score <= 40

```
SELECT student_id, subject, score
FROM scores
WHERE score >= 80;
```

7.8 Logical operators: AND, OR, NOT

Logical operators combine conditions. AND means both conditions must be true. OR means at least one condition must be true. NOT reverses a condition.

```
-- Level 9 students who live in Dhaka
SELECT first_name, last_name
FROM students
WHERE level = 9 AND city = 'Dhaka';

-- Students who live in Dhaka or Sylhet
SELECT first_name, last_name, city
FROM students
WHERE city = 'Dhaka' OR city = 'Sylhet';

-- Students who are not in level 9
SELECT first_name, last_name, level
FROM students
WHERE NOT level = 9;
```

Parentheses make logic clearer

When a WHERE clause has several AND and OR conditions, use parentheses to show your intended logic. This avoids confusing results.

7.9 BETWEEN

BETWEEN checks whether a value is within a range. In SQL, BETWEEN is usually inclusive, meaning it includes the start and end values.

```
SELECT student_id, subject, score
FROM scores
WHERE score BETWEEN 70 AND 90;
```

7.10 IN

IN checks whether a value is in a list of possible values. It is cleaner than writing many OR conditions.

```
SELECT first_name, last_name, city
FROM students
WHERE city IN ('Dhaka', 'Khulna');
```

7.11 LIKE and wildcards

LIKE searches for patterns in text. The percent sign (%) means any number of characters. The underscore (_) means exactly one character.

Pattern	Meaning	Example match
'A%'	Starts with A	Amina
'%an%'	Contains an	Rahman
'%a'	Ends with a	Rina
'_ina'	Exactly one character before ina	Rina


```
SELECT first_name, last_name
FROM students
WHERE first_name LIKE 'A%';
```

7.12 ORDER BY

ORDER BY sorts query results. ASC means ascending order. DESC means descending order. If you do not say ASC or DESC, many DBMSs use ascending order by default.

```
SELECT first_name, last_name, city
FROM students
ORDER BY city ASC;

SELECT student_id, subject, score
FROM scores
ORDER BY score DESC;
```

7.13 LIMIT or TOP

LIMIT restricts how many rows are returned. It is useful when a table has many rows and you only want to see a few. Some DBMSs use TOP or FETCH FIRST instead of LIMIT.

```
SELECT student_id, subject, score
FROM scores
ORDER BY score DESC
LIMIT 3;
```

This query shows the top three scores after sorting scores from highest to lowest.

7.14 The order of clauses

For the query types in this module, a common clause order is SELECT, FROM, WHERE, ORDER BY, and LIMIT. Later, GROUP BY and HAVING are added for summary queries.

```
SELECT column_names
FROM table_name
WHERE condition
ORDER BY column_name
LIMIT number_of_rows;
```

Key takeaways

- SELECT retrieves data from a table.
- WHERE filters rows before they are shown.
- AND, OR, and NOT combine conditions.
- LIKE searches text patterns using wildcards.
- ORDER BY sorts results, and LIMIT restricts the number of rows shown.

Module 8: SQL Functions

Learning goals

- Explain what a SQL function does.
- Use simple string, numeric, and date functions.
- Use aggregate functions such as COUNT, SUM, AVG, MIN, and MAX.
- Understand that function names can vary between DBMSs.

8.1 What is a function?

A function is a built-in operation that takes input, does work, and returns an output. For example, UPPER('Amina') returns 'AMINA'. Functions help clean, transform, calculate, and summarize data.

```
SELECT UPPER('Amina') AS uppercase_name;
```

Input	Function	Output
'Amina'	UPPER	'AMINA'
88.756	ROUND	89 or 88.76 depending on settings
Several score rows	AVG	Average score

SQL dialect note

Function names are not perfectly identical in every DBMS. For example, one system may use LENGTH while another may use LEN. The concept is more important than memorizing every variation at the beginning.

8.2 String functions

String functions work with text values. They can change capitalization, join text, remove extra spaces, or take part of a string.

Function	Purpose	Example
UPPER	Converts text to uppercase	UPPER('Amina') returns 'AMINA'
LOWER	Converts text to lowercase	LOWER('DHAKA') returns 'dhaka'
LENGTH	Counts characters	LENGTH('Amina') returns 5
TRIM	Removes extra spaces at beginning/end	TRIM(' Amina ') returns 'Amina'
SUBSTRING	Returns part of text	SUBSTRING('Amina', 1, 3) may return 'Ami'
CONCAT	Joins text together	CONCAT('Amina', ' ', 'Rahman')

```
SELECT
  first_name,
  UPPER(first_name) AS name_uppercase,
  LENGTH(first_name) AS name_length
FROM students;
```

AS gives a temporary name to the result column. This temporary name is called an alias. It makes the output easier to read.

8.3 Concatenation

Concatenation means joining text values together. It is often used to display a full name from first_name and last_name. The exact syntax can differ between DBMSs.

```
-- Common in MySQL and some other systems
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM students;

-- Common in SQLite and PostgreSQL
SELECT first_name || ' ' || last_name AS full_name
FROM students;
```

8.4 Numeric functions

Numeric functions work with numbers. They can round values, remove negative signs, or move numbers up or down to whole numbers.

Function	Purpose	Example
ROUND	Rounds a number	ROUND(88.756, 1) returns 88.8
CEILING or CEIL	Rounds up to next whole number	CEILING(88.1) returns 89
FLOOR	Rounds down to previous whole number	FLOOR(88.9) returns 88
ABS	Returns positive distance from zero	ABS(-5) returns 5

```
SELECT
  score,
  ROUND(score / 10.0, 1) AS score_out_of_10
FROM scores;
```

8.5 Date functions

Date functions work with dates and times. They can show the current date, extract parts of a date, or add time to a date. Date functions vary a lot by DBMS, so always check the syntax for the system you are using.

Need	Possible SQL function or idea
Current date	CURRENT_DATE
Current date and time	CURRENT_TIMESTAMP
Year from a date	EXTRACT(YEAR FROM exam_date) or YEAR(exam_date)
Month from a date	EXTRACT(MONTH FROM exam_date) or MONTH(exam_date)
Add days to a date	DATE_ADD, exam_date + INTERVAL, or date modifiers

```
SELECT CURRENT_DATE AS today;
```

```
SELECT score_id, subject, exam_date
FROM scores
WHERE exam_date = '2026-01-15';
```

8.6 Aggregate functions

Aggregate functions summarize many rows into one result. They are very important for reports. For example, a teacher may not only want to see every score, but also the average score, highest score, and number of submitted scores.

Aggregate function	Meaning	Example question
COUNT	Counts rows or non-NULL values	How many scores are stored?
SUM	Adds numeric values	What is the total of all scores?
AVG	Finds the average of numeric values	What is the average score?
MIN	Finds the smallest value	What is the lowest score?
MAX	Finds the largest value	What is the highest score?

```
SELECT COUNT(*) AS number_of_scores
FROM scores;
```

```
SELECT AVG(score) AS average_score
FROM scores;
```

```
SELECT MIN(score) AS lowest_score, MAX(score) AS highest_score
FROM scores;
```

8.7 COUNT(*) vs COUNT(column)

COUNT(*) counts rows. COUNT(column_name) counts rows where that column is not NULL. This difference matters when some values are missing.

Expression	What it counts
COUNT(*)	All rows returned by the query
COUNT(email)	Only rows where email is not NULL

8.8 Combining functions with WHERE

WHERE can filter rows before an aggregate function calculates the result. For example, the following query finds the average only for Mathematics scores.

```
SELECT AVG(score) AS average_math_score
FROM scores
WHERE subject = 'Mathematics';
```

8.9 Common beginner mistakes with functions

Mistake	Why it is a problem	Better habit
Using a text function on a number without reason	The result may be confusing	Choose a function that matches the data type
Forgetting that functions can change display only	UPPER(name) does not permanently change stored data	Use UPDATE only if permanent change is needed
Expecting all DBMSs to use same function names	Syntax can differ	Check your DBMS documentation or class notes
Ignoring NULL values in aggregates	Some aggregates skip NULL values	Understand what missing values mean

Key takeaways

- Functions take input and return output.
- String functions work with text; numeric functions work with numbers; date functions work with dates.
- Aggregate functions summarize many rows.
- COUNT(*) counts rows, while COUNT(column) counts non-NULL values in that column.
- SQL function names may differ between DBMSs.

Module 9: Grouping Data

Learning goals

- Explain why GROUP BY is needed.
- Use GROUP BY with aggregate functions.
- Use HAVING to filter groups.
- Recognize common grouping mistakes.
- Create simple summary reports from a table.

9.1 Why grouping is useful

Aggregate functions can summarize all rows into one result. But sometimes we need separate summaries for different categories. For example, a teacher may want the average score for each subject, not only the average score for the whole table. GROUP BY creates groups of rows that share the same value.

```
-- Average score for the whole scores table
SELECT AVG(score) AS overall_average
FROM scores;

-- Average score for each subject
SELECT subject, AVG(score) AS average_score
FROM scores
GROUP BY subject;
```

9.2 How GROUP BY works

GROUP BY puts rows into groups based on one or more columns. Then aggregate functions calculate one result for each group.

subject	score values in that group	AVG(score)
English	79, 84, 92	85.0
Mathematics	88, 72, 65	75.0
Science	91, 77	84.0

```
SELECT subject, AVG(score) AS average_score
FROM scores
GROUP BY subject;
```

9.3 GROUP BY with COUNT

COUNT is often used with GROUP BY to count how many rows are in each category.

```
SELECT subject, COUNT(*) AS number_of_scores
FROM scores
GROUP BY subject;
```

subject	number_of_scores
English	3
Mathematics	3
Science	2

9.4 GROUP BY with MIN and MAX

You can use more than one aggregate function in the same grouped query. This is useful for reports.

```
SELECT
  subject,
  MIN(score) AS lowest_score,
  MAX(score) AS highest_score,
  AVG(score) AS average_score
FROM scores
GROUP BY subject;
```

9.5 GROUP BY with WHERE

WHERE filters rows before the grouping happens. For example, the following query only considers scores from 2026-01-15 and then groups the remaining rows by subject.

```
SELECT subject, AVG(score) AS average_score
FROM scores
WHERE exam_date = '2026-01-15'
GROUP BY subject;
```

9.6 HAVING

HAVING filters groups after GROUP BY has created the groups. WHERE filters individual rows before grouping. HAVING filters summary results after grouping. This difference is important.

```
SELECT subject, AVG(score) AS average_score
FROM scores
GROUP BY subject
HAVING AVG(score) >= 80;
```

This query shows only subjects whose average score is at least 80. The condition uses AVG(score), so it must be checked after groups are formed.

Clause	Filters what?	Example
WHERE	Individual rows before grouping	WHERE subject = 'English'
HAVING	Groups after aggregation	HAVING AVG(score) >= 80

9.7 Grouping by more than one column

You can group by more than one column. The database then creates groups for each combination of values. For example, grouping by subject and exam_date gives one group for each subject on each exam date.

```
SELECT subject, exam_date, COUNT(*) AS number_of_scores
FROM scores
GROUP BY subject, exam_date
ORDER BY exam_date, subject;
```

9.8 Common grouping mistake

A common beginner mistake is selecting a normal column that is not in GROUP BY and is not inside an aggregate function. In a grouped query, each selected column must either be part of the grouping or be summarized with an aggregate function.

```
-- Problematic idea
SELECT subject, student_id, AVG(score)
FROM scores
GROUP BY subject;
```

This is confusing because each subject group can contain many student_id values. Which student_id should the database show for the group? A better query either removes student_id or groups by subject and student_id.

```
-- Better: summary by subject only
SELECT subject, AVG(score) AS average_score
FROM scores
GROUP BY subject;

-- Better: summary by subject and student
SELECT subject, student_id, AVG(score) AS average_score
FROM scores
GROUP BY subject, student_id;
```

9.9 Complete report example

The following query creates a subject report. It counts the number of scores, finds the lowest and highest score, and calculates the average score for each subject. It then shows only subjects with at least two scores and sorts the strongest average first.

```
SELECT
  subject,
  COUNT(*) AS number_of_scores,
  MIN(score) AS lowest_score,
  MAX(score) AS highest_score,
  ROUND(AVG(score), 1) AS average_score
FROM scores
GROUP BY subject
HAVING COUNT(*) >= 2
ORDER BY average_score DESC;
```

subject	number_of_scores	lowest_score	highest_score	average_score
English	3	79	92	85.0
Science	2	77	91	84.0
Mathematics	3	65	88	75.0

9.10 Logical order of a grouped query

SQL is written in one order, but it is helpful to understand the logical order. The database first identifies the table, filters rows, forms groups, filters groups, chooses result columns, sorts, and then limits output.

Logical step	Clause	Question it answers
1	FROM	Which table are rows coming from?
2	WHERE	Which individual rows should be kept?
3	GROUP BY	How should rows be grouped?
4	HAVING	Which groups should be kept?
5	SELECT	Which columns or calculations should be displayed?
6	ORDER BY	How should results be sorted?
7	LIMIT	How many rows should be shown?

Key takeaways

- GROUP BY creates summaries for categories or combinations of categories.
- Aggregate functions calculate one result for each group.
- WHERE filters rows before grouping; HAVING filters groups after grouping.
- Every selected column in a grouped query should be grouped or aggregated.
- Grouped reports are useful for averages, counts, totals, minimums, and maximums.

Suggested answers to selected practice questions

These are sample answers. A correct answer may be worded differently if the meaning is the same.

Topic	Sample answer
Data vs information	88 is data. "Amina scored 88 in Mathematics" is information because context is added.
Primary key	A primary key uniquely identifies each row, such as student_id in students.
Foreign key	A foreign key points to a primary key in another table, such as scores.student_id.
Safe UPDATE	UPDATE students SET city = 'Dhaka' WHERE student_id = 2;
Find level 10 students	SELECT first_name, last_name FROM students WHERE level = 10;
Scores above 85	SELECT student_id, subject, score FROM scores WHERE score > 85;
Average by subject	SELECT subject, AVG(score) FROM scores GROUP BY subject;
WHERE vs HAVING	WHERE filters rows before grouping. HAVING filters groups after aggregation.

Beginner glossary

Term	Meaning
Aggregate function	A function that summarizes many rows, such as COUNT or AVG.
Attribute	A fact about an entity. In a table, it usually becomes a column.
Cardinality	The number pattern of a relationship, such as one-to-many.
Column	A vertical part of a table that stores one kind of fact.
Constraint	A database rule that protects data quality.
Database	An organized collection of related data.
DBMS	Software that manages databases.
DDL	SQL commands that define structure, such as CREATE TABLE.
DML	SQL commands that change data, such as INSERT or UPDATE.
Entity	Something we store data about, such as Student or Book.
ER diagram	A diagram that shows entities, attributes, and relationships.
Foreign key	A column that refers to a primary key in another table.
GROUP BY	A clause that groups rows for summary calculations.
HAVING	A clause that filters groups after aggregation.
NULL	A missing, unknown, or unavailable value.
Primary key	A column or columns that uniquely identify each row.
Query	A request sent to a database, often using SELECT.
Row	A horizontal record in a table representing one item.
SQL	Structured Query Language, used to communicate with relational databases.
Table	A structure that stores rows and columns about one kind of thing.